

LABORATORIO DI INGEGNERIA DEI SISTEMI SOFTWARE

Introduction

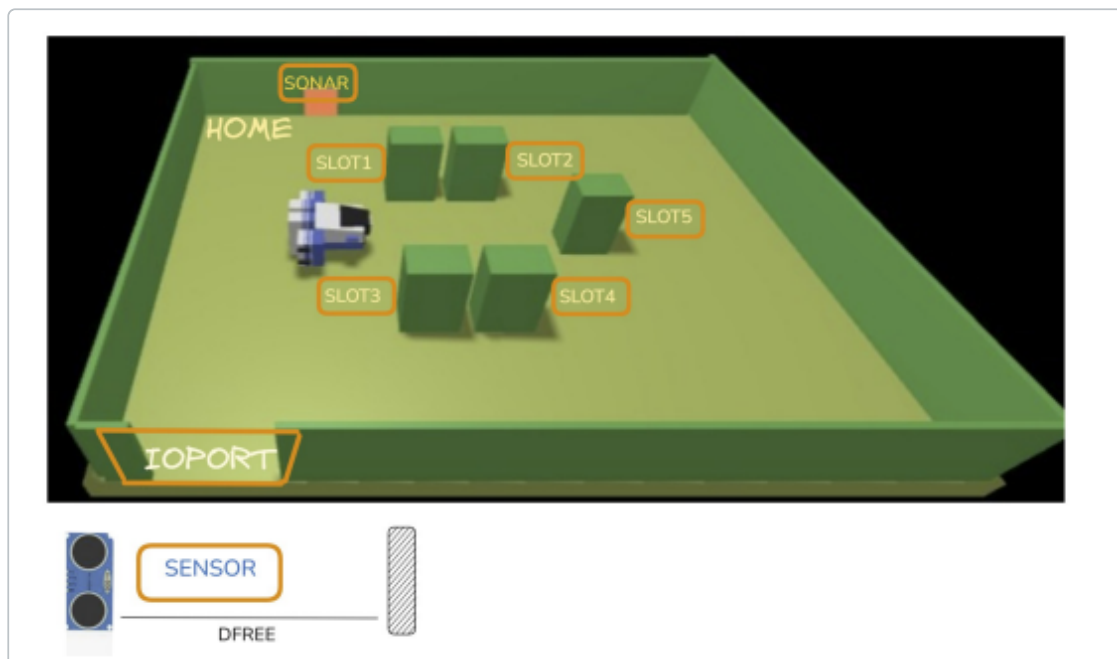
Questo documento descrive il lavoro svolto nello **Sprint 1** per lo sviluppo del sistema **cargoservice**. L'obiettivo principale di questo incremento è l'implementazione della logica di controllo centrale del servizio (l'orchestratore QActor `cargoservice`) e il superamento dell'**Abstraction Gap** per il pilotaggio del robot virtuale **RobotSmart26**.

Requirements

Il testo completo dei requisiti fornito dal committente è disponibile al seguente link: <Tema2026.html> (../Tema2026.html).

A **Maritime Cargo shipping company** (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a **Differential Drive Robot** (from now, called **cargorobot**).

The **hold** is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides 4 **slots** to store the containers and a slot named **slot5**.



In the picture above:

- The **slots1-4** depict the hold areas reserved to store one **container** each,

- The **slots5** depicts an area, where the **cargorobot** must temporarily store a container, before to place it in one of the **slots1-4**. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.
- The **IOPort** is a device with a **pushbutton** and a **display**. The **pushbutton** is pressed by the customer in order to to send a request to load a container on the cargo. The **display** is used to show the answer to the request and to show the current state of the **hold**.
- The **sensor** associated to the **IOPort** is a device (a **sonar**) used to detect the presence of a container, when it measures a distance D , such that $D < D_{FREE}/2$, during a reasonable time (e.g. 3 secs).

TF2026 Requirements

The company asks us to build service named **cargoservice** that should work as follows.

The **cargoservice** is able to receive a **request to load** a container sent by some customer by using the **pushbutton** of the **IOPort**.

- It sends the answer `retrylater`, if the **IOPort** is currently occupied by a container or if the system is **Out of service**
- It rejects the request when the hold is already full, i.e. the `slots1-4` are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the **request to load** is accepted, the customer must move the container in the **sensor** area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the **cargoservice** uses the **cargorobot** to move the container from the **IOPort** to the **slot5** (for marking the container) and then to the reserved slot.

The service must also show on the **display** on the **IOPort**:

- the current state of the **hold**
- the message '**Service working**', when it is all is going well

- the message '**Out of service**' if the **sonar sensor** measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the **sonar**).

Requirement analysis | Vocabolario

Nello Sprint 0 sono state chiarite le entità e mappate sui concetti formali di QActor, Contesto e Messaggi. La struttura a 3 contesti definita nello Sprint 0 rimane la nostra architettura logica di riferimento. In questo sprint implementeremo i primi due attori reali (**cargoservice** e **cargorobot**) mantenendo mockati gli input distribuiti (sonar e led) e l'interfaccia utente (WebGUI).

Problem analysis

Stato della Stiva (slots1-4 e slot5)

L'orchestratore deve memorizzare se ciascuno dei quattro slot di stoccaggio (**slots1-4**) è libero o occupato, oltre ad attendere la marcatura nello **slot5**. Poiché questo stato è puramente informativo e interno alla logica di business di **cargoservice**, viene modellato come un insieme di variabili booleane all'interno dello stato dell'attore QActor **cargoservice** (es. **Slot1Occupied**, **Slot2Occupied**, ecc.). Non vi è la necessità architetturalmente di introdurre un QActor separato per la stiva.

Risoluzione dell'Abstraction Gap per RobotSmart26

Il robot virtuale **RobotSmart26** (che risiede nel contesto esterno **ctxrobotsmart** alla porta **8020**) risponde a comandi cartesiani e di basso livello. Le nostre tappe logiche sono definite come costanti nominali: **ioport**, **slot5**, **slot1**, **slot2**, **slot3**, **slot4** e **home**. Esiste un evidente **abstraction gap** tra queste destinazioni semantiche e i comandi cartesiani richiesti dal robot.

Questo divario viene risolto all'interno del QActor **cargorobot**, che funge da driver di traduzione. Il **cargorobot** mantiene internamente una mappatura statica per tradurre le posizioni logiche in coordinate reali (X, Y) all'interno della griglia della stanza virtuale:

- **home** → (0,0)
- **ioport** → (4,0)
- **slot5** → (4,3)
- **slot1** → (1,1)
- **slot2** → (1,4)

- `slot3` → `(3,2)`
- `slot4` → `(3,4)`

Quando riceve la richiesta `move_robot(DEST)` da `cargoservice`, il `cargorobot` traduce `DEST` nelle coordinate corrispondenti e invia al `RobotSmart26` la richiesta formale di movimento:

```
Request moverobot : moverobot(TARGETX, TARGETY, STEPTIME)
```

Il `cargorobot` attende la risposta del robot (`moverobotdone` o `moverobotfailed`) e risponde a sua volta a `cargoservice` con il risultato della movimentazione.

Analisi dell'interazione distributiva

Le interazioni coinvolgono attori distribuiti su diversi nodi di rete. La comunicazione tra l'orchestratore e il driver del robot avviene tramite uno schema **Request-Reply sincrono**, in modo da bloccare la logica fino all'effettivo completamento del movimento fisico. La marcatura del container sullo `slot5` viene invece attivata tramite un **messaggio asincrono di Dispatch** (`start_marking`) e notificata di ritorno tramite un Dispatch di risposta (`marking_done`).

Test plans

Per validare la correttezza del comportamento del sistema nello Sprint 1, dobbiamo definire dei test automatizzati che avviino il sistema e simulino l'interazione del cliente. I test saranno scritti in Kotlin/Java tramite **JUnit 4**.

Test Funzionale 1: Carico Rifiutato (Stiva Piena)

Stato: Il `cargoservice` registra che gli slot da 1 a 4 sono occupati.

Azione: Invio richiesta `load` tramite CoAP.

Risultato Atteso: Il sistema risponde immediatamente con il messaggio `retrylater` o `reject` senza attivare la movimentazione robotica.

Test Funzionale 2: Accettazione Carico (Happy Path)

Stato: Stiva parzialmente vuota e sistema pienamente funzionante.

Azione: Si invia una richiesta `load` a stiva vuota. Si verifica che la risposta indichi lo slot assegnato (es. `slot1`). Si simula il rilevamento del container inviando il messaggio del sonar. Si osserva (via CoAP) che il robot esegua la sequenza completa e ritorni in `home`, occupando lo `slot1`.

Project

La progettazione e l'implementazione reale del codice dello Sprint 1 sono disponibili nei seguenti file:

- Modello eseguibile reale: `sprint1/src/cargoservice.qak`
(<https://github.com/riccardocar99/cargoservice-/blob/main/sprint1/src/cargoservice.qak>)
- Codice del test automatico: `sprint1/src-test/TestSprint1.kt`
(<https://github.com/riccardocar99/cargoservice-/blob/main/sprint1/src-test/TestSprint1.kt>)

Testing

I test automatici JUnit sono stati eseguiti con successo. L'output del comando `./gradlew test` conferma che tutti i test JUnit definiti (Happy Path e rifiuto stiva piena) passano regolarmente, validando la logica dello Sprint 1.

```
> Task :test
test.kotlin.TestSprint1 > testHappyPath PASSED
```

```
BUILD SUCCESSFUL in 10s
```

Deployment

Per eseguire il sistema nello Sprint 1:

1. Avviare il container Docker del robot virtuale **RobotSmart26** sulla porta `8020`.
2. Posizionarsi nella directory `sprint1` ed eseguire il comando `./gradlew run` per avviare il servizio principale.

Maintenance

(N/A per Sprint 1)

Luigi

Riccardo

Simone



GIT repo: <https://github.com/riccardocar99/cargoservice->
(<https://github.com/riccardocar99/cargoservice->)